答辩问答手册

《基于 Dify 工作流的智能客服助手与用户情感分析系统》

共 5 组 30 问，每题一句结论 + 支撑事实，便于临场组织语言

一、架构与选型

Q 为什么用 Dify，而不是自己写后端？

工作流可视化、知识库内建、调用可追溯，改流程不用改代码。代价是出问题时要靠平台日志定位，所以我把每个节点的耗时都抓出来做了对照。

Q 模型为什么要本地部署？

对话里会出现订单号、地址这类信息，本地推理保证数据不出内网；同时断网也能演示。但纯 CPU 的算力有限，所以本地模型定位是「兜底 + 对比」，不是主力。

Q 那主力为什么最后用了云端 qwen-plus？

20 次压测的数据说话：qwen-plus 中位数 1.75 秒、零超时；GLM-4-Flash 有 25% 的长尾落在 10 秒以上。本地 1.5B 单轮 0.62 秒很快，但长回答和质量不够。本地模型保留，随时可切回去演示。

Q 为什么不用 vLLM 提速？

vLLM 依赖较新的指令集，纯 CPU 上没有优势，镜像也大。llama.cpp 更轻，且能直接吃 Ollama 已缓存的权重。

Q 为什么选 GGUF 而不是 PyTorch 权重？

这台虚拟机只有 9.2 GB 内存，PyTorch 加载 1.5B 后逼近上限。GGUF Q4 权重 986 MB、运行时 1.8 GB，留足余量。

Q MCP 是什么？为什么需要它？

Model Context Protocol，把「模型能调用的工具」标准化。模型本身不能直连数据库，MCP 服务把 SQL 执行包装成一个工具，Dify 里当插件用。这也是任务 8 的要求。

Q 为什么 MCP 最后没留在微信主链路上？

实测 Dify 调 MCP 有约 4.5 秒的固定尖峰，而 MCP 服务自身只花 7 毫秒 —— 尖峰在 Dify 的插件加载层。把落库改成 Flask 直接连库后链路稳定，MCP 仍保留用于任务 8 的能力演示。

Q 三台机器为什么这么分工？

模型与编排需要大内存多核，放虚拟机；微信回调必须能访问内网 Dify 且跑签名校验，放 Windows；公网只做端口转发，2 核 2G 足够，也把攻击面收在一台可随时重建的机器上。

二、部署与运维

Q Xinference 为什么用 --network host？

bridge 模式下 9997 曾出现「容器内 200、宿主机 000」的端口映射失效，排查到 iptables 的 DOCKER 链被冲掉。改用 host 网络直接绕开转发。

Q --shm-size=4g 是干什么的？

容器默认 /dev/shm 只有 64 MB，Xinference 的 supervisor 子进程申请共享内存失败会直接退出，表现就是容器刚起来就 Exited。

Q 健康检查那三个环境变量是什么？

重试次数、重试间隔、单次超时。默认 5×5=25 秒就判集群不可用，而冷启动要 33~44 秒，所以每次都差 8 秒被误杀。放宽到 60 次后一次通过，没有改镜像。

Q Dify 这 11 个容器分别是什么？

api（后端）、worker + worker_beat（异步任务）、web（前端）、db（PostgreSQL）、redis（缓存）、weaviate（向量库）、sandbox（代码沙箱）、plugin_daemon（插件）、ssrf_proxy（出网代理）、nginx（入口）。

Q Docker 起不来时为什么先 restart docker？

它会重建 iptables 的 DOCKER 链和网桥路由，能一次性修好「容器在跑但端口不通」这类问题。

Q 为什么最后把 firewalld 停了？

firewalld 每次 reload 都会把 Docker 自己维护的链删掉，导致端口映射时通时不通。演示环境直接 disable，需要放行的端口用 iptables 持久化规则。

Q 公网是怎么暴露出去的？

云服务器跑 frps 监听 7000，本地 frpc 把 80 端口映射过去。微信只接受 80/443，而 Dify 在内网，所以必须有一层转口。

Q 机器重启后怎么自动恢复？

三层：容器用 restart=always 或 compose 的 restart 策略；Flask 与 frpc 用启动文件夹的 .cmd 拉起；Dify 用 docker compose up -d 一次性起齐。

三、性能与优化

Q 微信 5 秒是怎么分配的？

签名校验与解密 <10 ms；Dify 工作流 1.4~3.0 秒（超时阈值 4.8 秒）；落库放后台线程 15~30 ms；留 0.2~0.5 秒给网络与序列化。

Q 为什么要做预热？

首次请求实测 20.67 秒，是 worker 冷启 + 模型首 token。Flask 启动时发一次预热请求，把这段开销挪到启动阶段。

Q 本地模型 0.62 秒，为什么还说它慢？

那是一次 80 token 的短回答。回答变长、上下文变多时线性增长；而且 1.5B 的答案质量不足以支撑客服场景。

Q 延迟排查的顺序是什么？

先看宿主机频率（降频那次差了 9.6 倍），再看线程与批大小，然后才动量化与模型，最后才是改架构。

Q 20 次压测是怎么做的？

固定问题、固定账号，脚本连续调用 Dify 接口并逐节点记录耗时，输出中位数与超时率，避免只看一次结果。

Q 长尾抖动主要来自哪里？

两个来源：一是模型服务端 P95（换 qwen-plus 后消失），二是 Dify 插件闲置被卸载后重新加载（改 Flask 直连后绕开）。

四、业务与算法

Q 知识库是怎么切分的？

按 Markdown 标题切（分隔符 \n##），每段 500 字、重叠 50 字。这样「型号 + 参数 + 话术」各自成块，召回更准。

Q 为什么 TopK 取 4？

取 1 时只命中型号对比段，缺失参数；取 4 能同时覆盖参数段与话术段，再大就会引入无关内容拉低答案精度。

Q 提示词里 {{#context#}} 是什么坑？

Dify 的检索结果不会自动注入提示词，必须显式写这个占位符。不写的话检索白做，模型只能自由发挥。

Q 情感分析为什么从 82.6% 负面改到 40.7%？

SnowNLP 用商品评论训练，对疑问句一律给极低分，而客服对话一半以上是问句。修正方案是先匹配强词、把疑问句归中性、只对剩余样本用模型并收紧阈值。

Q 正面只有 1.12%，是不是算错了？

没有。全语料里「谢谢」「满意」等正向表达出现 0 次 —— 这个入口只承接售后咨询，用户不会在这里表达满意。这是语料属性，我如实写在报告里。

Q LDA 的主题数是怎么定的？

按一致性分数扫描，5 个主题时 0.437、交叉度低且能人工命名，更多主题会碎成同义簇。

Q 敏感消息是怎么处理的？

规则拦截分三级：辱骂（L3）不调模型直接划边界；威胁投诉（L2）承认权利并索要订单号；一般不满（L1）放行给模型安抚。拦截同样写库，作为舆情样本。

Q 大屏上为什么不显示年份？

按月聚合时把字段写成「1 月…12 月」，避免平台把它识别成日期字段自动补年份；趋势图的时间轴标注在副标题里说明。

五、临场与其他

Q 系统现在能跑吗？怎么演示？

能。演示五步：给公众号发「R40-A0额定功率多少」→ 秒回 2000W → 查库看新增记录 → 再发一条投诉类看拦截话术 → 打开大屏看趋势与情感分布。

Q 断网了还能用吗？

本地模型可以继续推理，知识库与工作流在本地也行；但微信收发与云端模型 API 需要网络。

Q 这个项目最大的技术难点是什么？

不是写工作流，而是三次被默认值坑：25 秒健康检查、64 MB 共享内存、空的 restart 策略。还有一次是宿主机降频，日志里完全看不出来。

Q 如果重做一次，你会改什么？

上 GPU 跑 vLLM 提吞吐；把多轮上下文做起来；给关键链路加自动化回归，这些默认值问题本该被测试拦住而不是靠排查。

Q 还有哪些没做完？

多轮记忆、语音消息（STT）、人工转接与工单联动、知识库的定时爬虫增量更新。

答题技巧：先给结论（一句话），再给一个具体数字或现象作为证据，最后补一句代价或反思。老师追问细节时，优先回答「我是怎么验证的」，而不是「应该是」。